



11io-systems

İşlemcinin Dış Dünyayla Konuşması

Hız Uçurumu (Speed Mismatch)

İşletim sisteminin I/O (Giriş/Çıkış) yönetimindeki en büyük kabusu, cihazlar arasındaki devasa hız farkıdır.

Eğer o süper hızlı işlemci, dış dünyadan gelecek bir veriyi boş boş beklerse, saniyede milyarlarca işlem yapma kapasitesi tamamen çöpe gider.

Bütün bu PDF'in amacı, işlemciyi bu bekleyiştten nasıl kurtaracağımızı anlatmaktır.

İletişim Yolları ve Denetleyiciler

İşlemci gidip de klavyeyle veya SSD ile doğrudan, yüz yüze konuşmaz. Ortada bir hiyerarşi ve çevirmenler vardır:

Veriyolları (Buses)

- Verilerin aktığı fiziksel kablo veya yollardır. Modern sistemlerde işlemci; ekran kartı (GPU) ve NVMe SSD gibi çok hızlı cihazlara **PCIe** (Express) yolları üzerinden *doğrudan* bağlanır.
- Daha yavaş olan USB, klavye ve ses cihazları ise güney köprüsü veya Platform Controller Hub (PCH) adı verilen alt istasyonlara bağlanır.

Denetleyici (Controller / Host Adapter)

- İşlemci sadece emri verir ("Şu veriyi diske yaz"). O emri alıp diskin motorunu döndüren, lazerini ayarlayan veya elektriksel sinyallere çeviren o küçük donanımsal beyne "Denetleyici" diyoruz.
- İşlemci her zaman cihazın kendisiyle değil, bu denetleyicinin üzerindeki "Kayıtçılarla" (Registers) muhatap olur.

İşlemcilerin Denetleyicilere Emir Vermesi

Peki, CPU bu denetleyicinin içindeki küçük hafıza kutularına (Register) nasıl yazacak? Bunun iki yöntemi vardır:

- **Eski ve Hantal Yöntem - PMIO (Port-Mapped I/O):** İşlemcinin sadece dış dünyayla konuşmak için kullandığı, RAM'den tamamen bağımsız, özel bir iletişim hattı (Port adresleri) vardır. CPU sadece bu hatta özel `in` ve `out` adı verilen Assembly komutlarıyla cihazlara emir gönderir. Ancak bu hat çok dardır (sadece 65.536 port adresi destekler) ve eski nesil yazıcı/klavye gibi yavaş cihazlarda kalmıştır.
- **Modern ve Sihirli Yöntem - MMIO (Memory-Mapped I/O):** Modern mühendisler harika bir illüzyon icat etmiştir. Cihazın denetleyicisini sanki **fiziksel RAM'in bir parçasıymış gibi** gösterirler!
 - Örneğin, ekran kartının komut merkezi RAM'deki `0xF0000000` adresine haritalanır.
 - İşlemci o adrese standart bir "Değişken Ata" (Load/Store) komutuyla bir veri yazar. Ama o veri RAM'e gitmez; donanım seviyesindeki Bus o veriyi anında havada kapıp ekran kartına yönlendirir!.
 - Ekstra hiçbir özel komuta gerek kalmaz, inanılmaz hızlıdır ve PCIe gibi modern donanımların hepsinde (örneğin NVMe SSD'lerde) bu kullanılır.

Emrin CPU'ya Dönüşü

İşlemci "Bana şu dosyayı diskten getir" emrini verdiğinde disk çalışmaya başlar. Peki ama işlemci, diskin o yavaş işini bitirdiğini nasıl öğrenecek?

Polling (Sürekli Sormak / Meşgul Bekleme)

- **Nasıl Çalışır?** İşlemci, cihazın durum kayıtçısına (Status Register) girer ve "Hazır mısın? Hazır mısın? Hazır mısın?" diye saniyede milyonlarca kez sorar (Busy-Wait).
- **Neden Kabustur?** Bir diskten verinin okunması yaklaşık 10 milisaniye sürer. Bu süre bizim için çok kısadır ama işlemci bu süre zarfında diske tam **1 milyar kez**

"Hazır mısınız?" diye sorabilir. Bu, işlemcinin gücünün ve zamanının %99.99'unun çöpe gitmesi, cihazın ısınıp pili sömürmesi demektir.

- **Ne Zaman İşe Yarar?** Sadece cihaz *inanılmaz derecede hızlıysa* veya modern yüksek performanslı ağ kartlarındaki gibi gecikmeye (latency) zerre kadar tahammülümüz yoksa kullanılır. Beklemektense o an orada hazır beklemek (spin atmak) daha mantıklı olur.

Interrupts

- **Nasıl Çalışır?** İşlemci emri cihaza verir ve anında kendi işine (örneğin arka planda Chrome'u çalıştırmaya) geri döner. Cihaz işini bitirdiğinde işlemciye elektriksel bir sinyal (Interrupt) yollar. İşlemci o anki işini dondurur, durumu kaydeder, cihazın verisini alır (Handler çalıştırır) ve sonra dondurduğu Chrome sekmesine geri döner.
- **Sorun Çıkaran Detay: Yönetim Kargaşası.** Bilgisayarda aynı anda fare, klavye, ağ kartı ve disk işlemciyi dürtmeye kalkarsa ne olur? İşletim sistemi karmaşayı önlemek için bir **Kesme Denetleyicisi (Interrupt Controller)** ve bir **Kesme Vektör Tablosu (Adres Defteri)** kullanır. Bu sayede kimin daha öncelikli olduğuna karar verir (Örn: Klavyeden girilen bir tuş, ağdan gelen bir paketten daha acildir) ve hangi cihaz için hangi kodu çalıştıracağını o tablodan bulur.

Hibrit Çözümler

- **Linux NAPI (Ağlar için Hibrit):** Ağ kartına binlerce paket yığıldığında, kart **sadece ilk paket için** bir kesme (interrupt) yollayıp işlemciyi uyandırır. İşlemci uyanır uyanmaz diğer kesmeleri geçici olarak sağlar (kapatır) ve o kuyruktaki tüm paketleri **Polling (Sürekli Sorma)** mantığıyla, kimse onu bölmeden hızla eritip temizler. Kuyruk bitince kesmeleri geri açar. Harika bir zekadır!
- **NVMe SSD Kapı Zili (Doorbell):** Yeni nesil M.2 SSD'lerde işlemci tek tek veri okumak yerine, 64 tane okuma emrini bir listeye yazar ve SSD'ye "Kapı Zilini Çalarak" (Ring Doorbell) "Toplu sipariş verdim, başla!" der. SSD 64 işlemi de bitirince tek bir kesme (interrupt) yollar. İşlemci tek bir kesmeyle 64 dosyanın sonucunu almış olur.

DMA (Doğrudan Bellek Erişimi)

İşlemciyi beklemekten kurtardık ama ortada çözülmesi gereken devasa bir "amelelik" sorunu daha var:

Programmed I/O ve Amelelik

Diskten RAM'e kopyalanacak 1 MB veri için CPU'nun 1 bayt 1 bayt okuma yazma yapması hamallıktır.

DMA Nedir?

Mühendisler işlemciyi bu veri kopyalama hamallığından kurtarmak için anakartın üzerine yepyeni, özel bir mini-işlemci eklerler. Buna **DMA Denetleyicisi** diyoruz. İşlemciyi bir şirketin "CEO'su", DMA'i ise "Lojistik Müdürü" olarak düşünebilirsin.

DMA Nasıl Çalışır?

1. **Emrin verilmesi:** İşlemci (CEO) diske ve DMA'e (Lojistik Müdürüne) sadece tek bir talimat verir: *"Diskteki şu adresteki dosyayı al, içindeki C baytlık veriyi RAM'deki X adresine taşı."*
İşin en güzel kısmı burasıdır: İşlemci emri verdiği saniye o işi tamamen unutur ve arka plandaki akıllıca işlerine (oyun oynatmaya, hesap yapmaya) geri döner.
2. **Hamallık:** DMA denetleyicisi sazi eline alır. Veriyolunu kullanarak diskten verileri alır ve doğrudan RAM'e yazar. Her baytı taşıdığında sayacı (C) birer birer düşürür (C=0 olana kadar bu ameleliği tek başına yapar). İşlemcinin ruhu bile duymaz.
3. **Görev Tamamlandı:** Taşıma işlemi tamamen bitip sayaç sıfırlandığında (C=0), DMA denetleyicisi işlemciye tek bir "Kesme" (Interrupt) sinyali gönderir: *"Patron, taşıma işini hallettim, dosyalar RAM'de hazır!"*

Döngü Çalma (Cycle Stealing)

Hayatta hiçbir şey tamamen bedava değildir. DMA'in verileri taşırken kullandığı Memory Bus, işlemcinin de kullandığı ana yoldur.

DMA veriyi RAM'e yazacağı o milisaniyelik anlarda otoyolu işgal ettiği için, işlemci o an RAM'e ulaşmak isterse kısıcık bir an beklemek zorunda kalır. DMA'in otoyolu

bu şekilde geçici olarak ele geçirmesine **Döngü Çalma (Cycle Stealing)** denir. İşlemciyi çok hafif yavaşlatsa da, onu 1 milyon kere git-gel yapmaktan kurtardığı için inanılmaz derecede kârlı bir takastır.

İşletim Sisteminin Çevirmenliği ve I/O Alt Sistemi

Dünyada on binlerce farklı fare, klavye, disk ve ekran kartı var. İşletim sistemi bu kaosu nasıl yönetiyor?

1. Uygulama I/O Arayüzü

İşletim sistemi, her cihaza özel kod yazmak yerine cihazları "Davranışlarına" göre 3-4 ana sınıfa böler ve hepsine standart bir kıyafet giydirir:

- **Blok Cihazları (Block Devices):** Veriyi büyük paketler (bloklar) halinde okuyup yazan cihazlardır (Örneğin Hard Diskler ve SSD'ler).
- **Karakter (Akış) Cihazları (Character Devices):** Veriyi harf harf, sırayla gönderen cihazlardır (Örneğin Klavye ve Fare).
- **Ağ Soketleri (Network Sockets):** Bağlantı ve paket mantığıyla çalışan cihazlardır (Modemler ve Ağ Kartları).

Sürücüler (Device Drivers), bu farklılıkları işletim sisteminden gizleyen "Çevirmenlerdir". İşletim sistemi fareye de "Oku" der, diske de "Oku" der; sürücü bunu cihazın anlayacağı elektrik sinyaline çevirir.

2. Beklemek ya da Beklememek

- **Bloklama (Blocking - Senkron):** En yaygınıdır. Program dosyayı ister ve veri gelene kadar **tamamen donar, uykuya dalar**. Veri gelince uyanır ve kaldığı yerden devam eder.
- **Bloklamasız (Non-blocking):** Program veriyi ister. Eğer veri o an hazır değilse, sistem programı dondurmaz; "Şu an boşum, sonra gel" der ve program sürekli veri geldi mi diye sorar.

- **Asenkron (Asynchronous):** En zekice olanıdır. Sen kargoyu sipariş edersin, kapıda beklemezsin (bloke olmazsın). Kendi işine bakarsın. Kargo geldiğinde kurye sana "Geldik" diye haber (Signal) verir.

3. Çekirdek I/O Hizmetleri: Performans Kurtarıcıları

- **Tamponlama (Buffering):** Klavye çok yavaş, işlemci çok hızlıdır. Klavyeden gelen harfler anında işlemciye gitmez, önce hafızadaki bir "Tampon" (Kutu) içine birikir. Kutu dolunca işlemci tek seferde alır. Çift Tamponlama (Double Buffering) hilesiyle kutulardan biri dolarken, işlemci diğer dolu kutuyu işler; böylece sistem hiç duraksamaz.
- **Önbellekleme (Caching):** Sık okunan verilerin (Örn: Sık girdiğin bir web sitesinin resimleri), yavaş disk yerine çok hızlı olan RAM'de (Önbellekte) bir kopyasının tutulmasıdır.
- **Kuyruklama (Spooling):** Yazıcı gibi "Aynı anda sadece tek bir kişiye" hizmet verebilen cihazlar için kullanılır. Sen bir çıktı gönderdiğinde, belge direkt yazıcıya değil, diskin içindeki bir kuyruğa (Spool) gider. Böylece yazıcı dolu olsa bile senin programın donmaz, belgeyi kuyruğa bırakıp işine devam eder.

4. I/O Yaşam Döngüsü

Farenle bir dosyaya tıkladın, neler oluyor? Bu süreç 6 muazzam adımdan oluşur:

1. Program bir "Sistem Çağrısı" (System Call) yaparak çekirdeğe iner.
2. Çekirdek bakar: "Bu dosya benim hızlı önbelleğimde (Cache) zaten var mı?" Varsa, diski hiç yormaz, veriyi hemen sana geri verir.
3. Yoksa, isteği cihazın sürücüsüne (Driver) iletir ve senin programını beklemeye (uykuya) alır.
4. Sürücü, donanım denetleyicisine komut verir. Donanım (Örn: DMA kullanarak) diskten veriyi okuyup RAM'e kopyalar.
5. Kopyalama bitince, donanım işlemciye bir "Kesme" (Interrupt) fırlatır: "İşlem tamam!".
6. Çekirdeğin Kesme İşleyicisi uyanır, verinin başarıyla geldiğini görür, uyuyan senin programını uyandırır ve veriyi ellerine teslim eder.

